5. Tusk 컨센서스 프로토콜

5.1 Tusk 프로토콜 개요

Tusk는 Narwhal 논문에서 제안된 원본 컨센서스 알고리즘으로, DAG 구조를 활용한 비잔틴 장애 허용 컨센서스 프로토콜입니다.

5.1.1 핵심 특징

- **리더 기반**: 각 라운드마다 리더를 선출하여 순서 결정
- **2단계 커밋**: 리더 선출과 커밋의 2단계 과정
- **확률적 종료**: 공통 코인을 사용한 확률적 합의
- **비동기 안전성**: 네트워크 지연에 관계없이 안전성 보장

5.1.2 프로토콜 구조

```
pub struct Tusk {
pub committee: Committee,
pub store: Arc<ConsensusStore>,
pub gc_depth: Round,
}
```

5.2 리더 선출 메커니즘

5.2.1 리더 선출 알고리즘

```rust
impl Tusk {
fn leader<'a>(
committee: &Committee,
round: Round,
dag: &'a Dag,
) -> Option<&'a (CertificateDigest, Certificate)> {
// 스테이크 가중치 기반 리더 선출
let leader = &committee.leader(round);
// 해당 라운드에서 리더의 인증서 반환
dag.get(&round).and_then(|x| x.get(leader))
}
}
```

5.2.2 리더 선출 특징

- **결정론적**: 동일한 라운드에서 모든 노드가 같은 리더 선출
- **스테이크 가중**: 높은 스테이크를 가진 노드가 리더가 될 확률 높음
- **라운드 시드**: 라운드 번호를 시드로 사용한 의사 랜덤 선택
- **공정성**: 장기적으로 모든 노드가 리더가 될 기회 보장

5.3 커밋 조건과 과정

5.3.1 커밋 조건 확인

```
fn  process_certificate(
&mut  self,
state: &mut ConsensusState,
consensus_index: SequenceNumber,
certificate: Certificate,
) -> StoreResult<Vec<ConsensusOutput>> {
let  round = certificate.round();
let  r = round - 1;
// 짝수 라운드에서만 리더 선출
if  r % 2 != 0 || r < 4 {
return Ok(Vec::new());
}
let  leader_round = r - 2;
let (leader_digest, leader) = match  Self::leader(&self.committee, leader_round,
Some(x) => x,
None => return Ok(Vec::new()),
};
// f+1 지지 확인
let  stake: Stake = state
.dag
.get(&(r - 1))
.expect("We should have the whole history by now")
```

```
.values()
.filter(|(_, x)| x.header.parents.contains(leader_digest))
.map(|(_, x)| self.committee.stake(&x.origin()))
.sum();
if  stake < self.committee.validity_threshold() {
return Ok(Vec::new());
}
// 커밋 수행
self.commit_leader_chain(state, leader, consensus_index)
}
```

5.3.2 커밋 과정 상세

1. **리더 식별**: r-2 라운드의 리더 인증서 찾기
2. **지지 확인**: r-1 라운드에서 f+1 이상의 지지 확인
3. **체인 구성**: 커밋할 리더들의 체인 구성
4. **순서 결정**: DAG 순회를 통한 트랜잭션 순서 결정
5. **상태 업데이트**: 커밋된 인증서들의 상태 업데이트

5.4 DAG 순서화 알고리즘

5.4.1 리더 체인 구성

```
// utils.rs에서 구현된 리더 순서화
```

```rust
pub fn order_leaders<F>(
committee: &Committee,
leader: &Certificate,
state: &ConsensusState,
leader_fn: F,
) -> Vec<Certificate>
where
F: Fn(&Committee, Round, &Dag) -> Option<&(CertificateDigest, Certificate)>,
{
let mut leaders = Vec::new();
let mut current = leader;
// 역방향으로 리더 체인 구성
loop {
leaders.push(current.clone());
// 이전 리더 찾기
let prev_round = current.round().saturating_sub(2);
if prev_round <= state.last_committed_round {
break;
}
match leader_fn(committee, prev_round, &state.dag) {
Some((_, prev_leader)) => current = prev_leader,
None => break,
}
```

```
}
leaders
}
```

5.4.2 DAG 평면화

```
pub fn order_dag(
gc_depth: Round,
leader: &Certificate,
state: &ConsensusState,
) -> Vec<Certificate> {
let mut ordered = Vec::new();
let mut visited = HashSet::new();
let mut stack = vec![leader.clone()];
// DFS를 통한 의존성 순서 결정
while let Some(cert) = stack.pop() {
if visited.contains(&cert.digest()) {
continue;
}
visited.insert(cert.digest());
ordered.push(cert.clone());
// 부모들을 스택에 추가
for parent_digest in &cert.header.parents {
```

```
if let Some(parent) = find_certificate_in_dag(parent_digest, state) {
if !visited.contains(parent_digest) {
stack.push(parent);
}
}
}
}
// 위상 정렬 순서로 반환
ordered.reverse();
ordered
}
```

5.5 안전성과 활성 보장

5.5.1 안전성 속성

- **합의 일관성**: 모든 정직한 노드가 같은 순서로 트랜잭션 커밋
- **유효성**: 커밋된 트랜잭션은 모두 유효한 트랜잭션
- **무결성**: 비잔틴 노드가 임의의 트랜잭션을 삽입할 수 없음

5.5.2 활성 보장

```
// 활성을 위한 최소 조건 확인
fn ensure_liveness(&self, state: &ConsensusState, round: Round) -> bool {
let certificates_in_round = state.dag.get(&round).map(|r| r.len()).unwrap_or(0)
```

```
let required_certificates = self.committee.quorum_threshold();

certificates_in_round >= required_certificates

}
```

5.6 성능 최적화

5.6.1 상태 제한

```
#[cfg(test)]

mod tests {

#[test]

fn state_limits_test() {

// DAG 크기가 제한된 범위 내에서 유지되는지 확인

let gc_depth = 12;

let rounds: Round = rand::thread_rng().gen_range(10..100);

// 최적 인증서로 테스트

let (certificates, _) = test_utils::make_optimal_certificates(

&committee, 1..=rounds, &genesis, &keys

);

for certificate in certificates {

tusk.process_certificate(&mut state, consensus_index, certificate).unwrap();

}

// DAG 크기가 6 라운드 이하로 유지되는지 확인
```

```
assert!(state.dag.len() <= 6);
}
}
```

5.6.2 메모리 관리

- **가비지 컬렉션**: 커밋된 인증서의 주기적 정리
- **상태 압축**: 불필요한 상태 정보 제거
- **캐시 최적화**: 자주 접근하는 데이터의 캐싱

5.7 장애 허용성

5.7.1 비잔틴 장애 모델

- **최대 f개 장애**: 전체 3f+1 노드 중 f개까지 비잔틴 장애 허용
- **임의 행동**: 비잔틴 노드의 임의적 행동에도 안전성 보장
- **네트워크 분할**: 일시적 네트워크 분할에도 복구 가능

5.7.2 복구 메커니즘

```
impl Tusk {
pub fn recover_from_failure(&mut self,
state: &mut ConsensusState
) -> Result<(), RecoveryError> {
// 마지막 커밋 지점부터 복구
let last_committed = state.last_committed_round;
```

```
// 누락된 인증서 식별 및 복구
self.sync_missing_certificates(last_committed)?;
// 상태 일관성 검증
self.verify_state_consistency(state)?;
Ok(())
}
}
```

5.8 성능 특성

5.8.1 지연시간 분석

- **최소 지연**: 4 라운드 (리더 선출 + 지지 + 커밋)
- **평균 지연**: 네트워크 조건에 따라 가변
- **최대 지연**: 네트워크 분할 시 무한정 지연 가능

5.8.2 처리량 특성

- **병렬 처리**: 여러 리더가 동시에 처리 가능
- **배치 효율**: 큰 배치 크기로 높은 처리량 달성
- **확장성**: 노드 수 증가에 따른 선형적 처리량 증가

Tusk 프로토콜은 이러한 메커니즘들을 통해 높은 성능과 강력한 안전성을 동시에 제공하는 컨센서스 알고리즘입니다.